

Medical Journey Log: A Family Care Tracker
Alex Frear
CST-452 Milestone 4: Development Phase (Coding and Testing)
Grand Canyon University
Instructor: Professor Amr Elchouemi
9/25/2025

Screencast Video 1 (Code Review) Link: [CST452 - Milestone 4 - Code Review](#)

Screencast Video 2 (Application Running) Link: [CST452 - Milestone 4 - App Running](#)

Github Link: [Milestone 4 Folder](#)

Table of Contents

1. Implementation Plan – Iteration 2	4
Implementation Plan Table	4
Iteration 2 Target Effort	4
Definition of Done	4
2. Requirements Traceability Matrix – Iteration 2.....	5
Traceability Matrix Table	5
3. Module Test Cases – Iteration 2.....	6
Test Case TC-005 – Edit Child.....	6
Edit Child (Before Update).....	7
Edit Child (Successful Update).....	8
Edit Child (Validation Error).....	9
Test Case TC-006 – Delete Child (Guarded)	10
Delete Child (Successful Deletion).....	11
Delete Child (Deletion Blocked Due to Symptoms)	12
Test Case TC-007 – Edit Symptom.....	13
Edit Symptom (Before Update)	14
Edit Symptom (Successful Update).....	15
Edit Symptom (Validation Error)	16
Test Case TC-008 – Delete Symptom	17
Delete Symptom (Before Deletion)	18
Delete Symptom (Successful Deletion)	19
4. Unit Test Plan.....	20
Unit Test Strategy	20
Unit Test Cases.....	20
Entry and Exit Criteria	20
5. Integration Test Plan.....	21
Integration Test Strategy.....	21
Integration Test Cases	21
Entry and Exit Criteria	21
6. System Test Plan	22
System Test Strategy.....	22
System Test Cases	22
Entry and Exit Criteria	22
7. Test Case Specifications.....	23
TC-005: Edit Child	23
TC-006: Delete Child (Guarded)	23
TC-007: Edit Symptom.....	23
TC-008: Delete Symptom.....	23
8. Test Analysis Approval Form.....	24

Test Summary	24
9. Source Code Listing – Iteration 2.....	25
Updated Model Files	25
Child.cs	25
Symptom.cs	26
New Razor Pages.....	27
EditChild.cshtml.cs	27
DeleteChild.cshtml.cs	28
EditSymptom.cshtml.cs	29
DeleteSymptom.cshtml.cs.....	30
Updated Razor Page.....	31
ChildDetails.cshtml	31

1. Implementation Plan – Iteration 2

Iteration 2 of the Medical Journey Log project focuses on expanding functionality to include editing and deleting both children and symptoms, as well as enforcing validation rules through data annotations. This iteration builds on the foundational features implemented in Iteration 1 (logging children and logging symptoms) and represents the second major coding cycle for the project. The scope, tasks, and estimated effort are documented below.

Implementation Plan Table

ID	User Story	Tasks	Est (hrs)	Actual (hrs)	Status
F1	As a caregiver, I can edit a child so I can correct details.	• Add [Required]/[StringLength] to Child model • Create <code>EditChild</code> page • Show validation messages	2.0		Planned
F2	As a caregiver, I can delete a child if no symptoms exist.	• Create <code>DeleteChild</code> page • Add guard to block deletion if symptoms exist • Display error message	1.5		Planned
F3	As a caregiver, I can edit a symptom to fix mistakes.	• Add annotations to <code>Symptom</code> • Create <code>EditSymptom</code> page • Maintain association to <code>ChildId</code>	2.0		Planned
F4	As a caregiver, I can delete a symptom I added by mistake.	• Create <code>DeleteSymptom</code> page • Redirect back to Child Details	1.0		Planned
F5	As a caregiver, I see useful validation errors when I submit bad data.	• Add [Required], [StringLength], [Range] attributes • Ensure <code>ModelState.IsValid</code> checks work	1.0		Planned

Iteration 2 Target Effort: 7.5 hours

Definition of Done:

- All features F1–F5 implemented and tested
- Unit, Integration, and System Test Plans documented
- Test Case Specifications updated with TC-005 through TC-008
- Successful application demo recorded and linked

2. Requirements Traceability Matrix – Iteration 2

The following matrix maps Iteration 2 requirements to their corresponding design references, code implementation, and test cases. This extends the traceability matrix from Iteration 1 by covering new functionality implemented during the second coding cycle.

Traceability Matrix Table

Req ID	Requirement (Iteration 2)	Design Reference	Code Reference	Test Case(s)
F1	Caregiver can edit a child to correct details.	Child entity; Edit Child workflow	EditChild.cshtml, EditChild.cshtml.cs	TC-005
F2	Caregiver can delete a child if no symptoms exist.	Child entity; Delete flow with guard	DeleteChild.cshtml, DeleteChild.cshtml.cs	TC-006
F3	Caregiver can edit a symptom .	Symptom entity; Edit Symptom workflow	EditSymptom.cshtml, EditSymptom.cshtml.cs	TC-007
F4	Caregiver can delete a symptom .	Symptom entity; Delete Symptom flow	DeleteSymptom.cshtml, DeleteSymptom.cshtml.cs	TC-008
F5	Validation ensures bad/missing input is rejected with error messages.	Data Annotations on models	Child.cs, Symptom.cs	TC-005 to TC-008

3. Module Test Cases – Iteration 2

The following module-level test cases validate the new features developed in Iteration 2. Each case includes positive and negative paths to confirm both functional behavior and validation enforcement. Evidence for each test case will be provided via screenshots from the running application.

Test Case TC-005 – Edit Child

Field	Value
Objective	Verify caregiver can edit a child’s details, with validation for bad input.
Pre-Conditions / Data	Existing Child with Name = “Test Child” and DOB set.
Steps (Positive)	1. Navigate to /EditChild/{ChildId} 2. Change Name to “Updated Child” 3. Click Save.
Expected (Positive)	Redirect to /ChildDetails/{ChildId}; success alert displayed; DB updated with new Name.
Steps (Negative A)	Clear Name field and Save.
Expected (Negative A)	Validation error “Name is required.”; remain on page; DB unchanged.
Steps (Negative B)	Enter a Name longer than 80 characters and Save.
Expected (Negative B)	Validation error “Name cannot be longer than 80 characters.”; DB unchanged.
Evidence	Screenshots of success and validation error states.
Status	Pass / Fail

Edit Child (Before Update)

The screenshot shows a web browser window with the address bar displaying 'localhost:5209/EditChild/5'. The page title is 'MedicalJourneyLog'. The navigation bar includes links for 'Home', 'Log Symptom', 'Log Child', and 'Privacy'. The main content area is titled 'Edit Child' and contains a form with two input fields: 'Name' (containing 'Test Child 2') and 'Date of Birth' (containing '01/01/2022'). Below the fields are 'Save' and 'Cancel' buttons. The footer of the page shows '© 2025 - MedicalJourneyLog - [Privacy](#)'.

Figure 1 Screenshot showing the Edit Child page before changes are made. The form displays the existing Name and Date of Birth values for the selected child.

Edit Child (Successful Update)

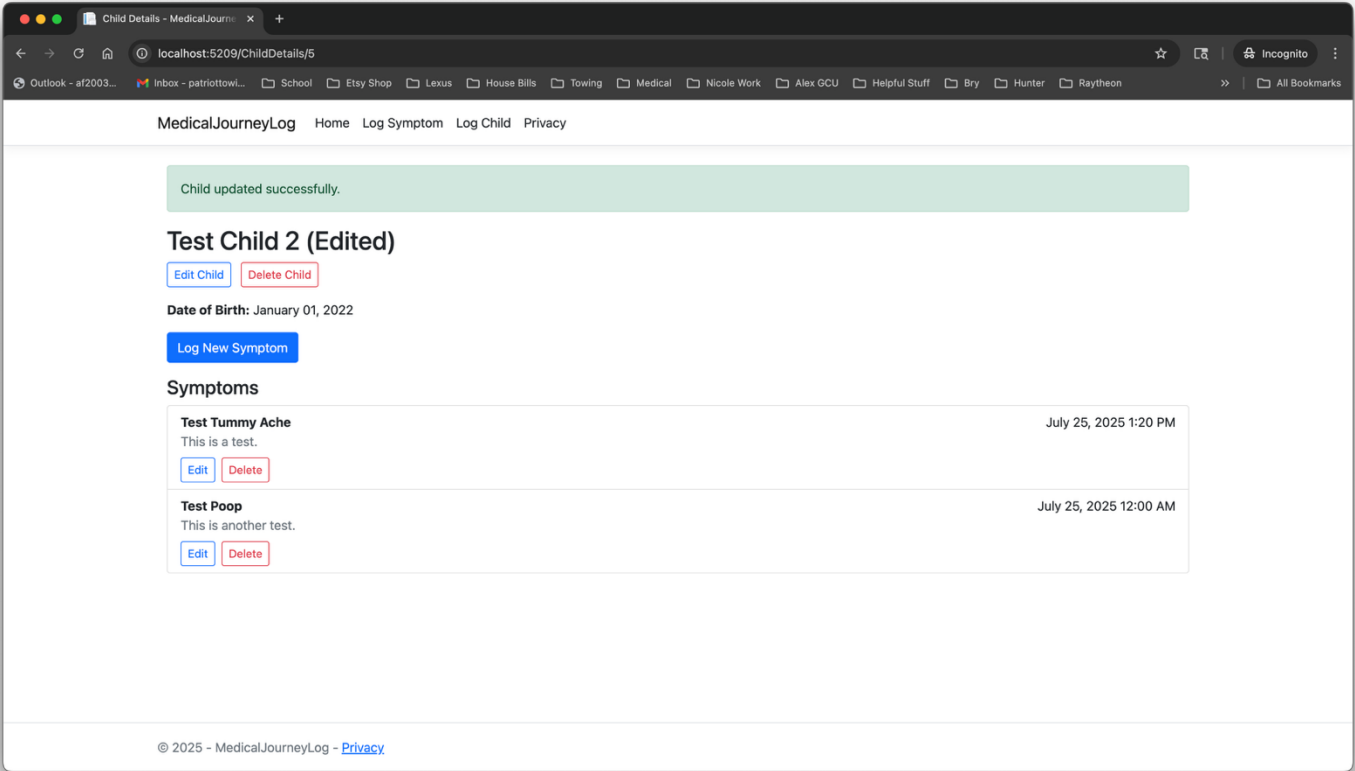


Figure 2 Screenshot showing the success alert after the child’s information is updated. The system confirms the change and redirects back to the Child Details page.

Edit Child (Validation Error)

The screenshot shows a web browser window with the address bar displaying 'localhost:5209/EditChild/5'. The page title is 'MedicalJourneyLog'. The navigation menu includes 'Home', 'Log Symptom', 'Log Child', and 'Privacy'. The main heading is 'Edit Child'. Below it, there is a 'Name' label and an empty text input field. A red error message, 'Name is required.', is displayed directly beneath the input field. Below the Name field is a 'Date of Birth' label and a date picker showing '01/01/2022'. At the bottom of the form are two buttons: 'Save' (in blue) and 'Cancel' (in grey). The footer of the page contains the text '© 2025 - MedicalJourneyLog - [Privacy](#)'.

Figure 3 Screenshot displaying server-side validation preventing submission of an empty Name field. The error message appears beneath the input, and no changes are saved to the database.

Test Case TC-006 – Delete Child (Guarded)

Field	Value
Objective	Verify caregiver can delete a child if no symptoms exist; block deletion if symptoms exist.
Pre-Conditions / Data	Case A: Child with 0 symptoms . Case B: Child with ≥1 symptom .
Steps (Case A)	1. Navigate to /DeleteChild/{ChildId} 2. Confirm Delete.
Expected (Case A)	Redirect to /Index ; success alert displayed; child removed from DB.
Steps (Case B)	1. Navigate to /DeleteChild/{ChildId} 2. Confirm Delete.
Expected (Case B)	Redirect back to /ChildDetails/{ChildId} ; error alert shown; DB unchanged.
Evidence	Screenshots showing allowed delete and blocked delete.
Status	Pass / Fail

Delete Child (Successful Deletion)

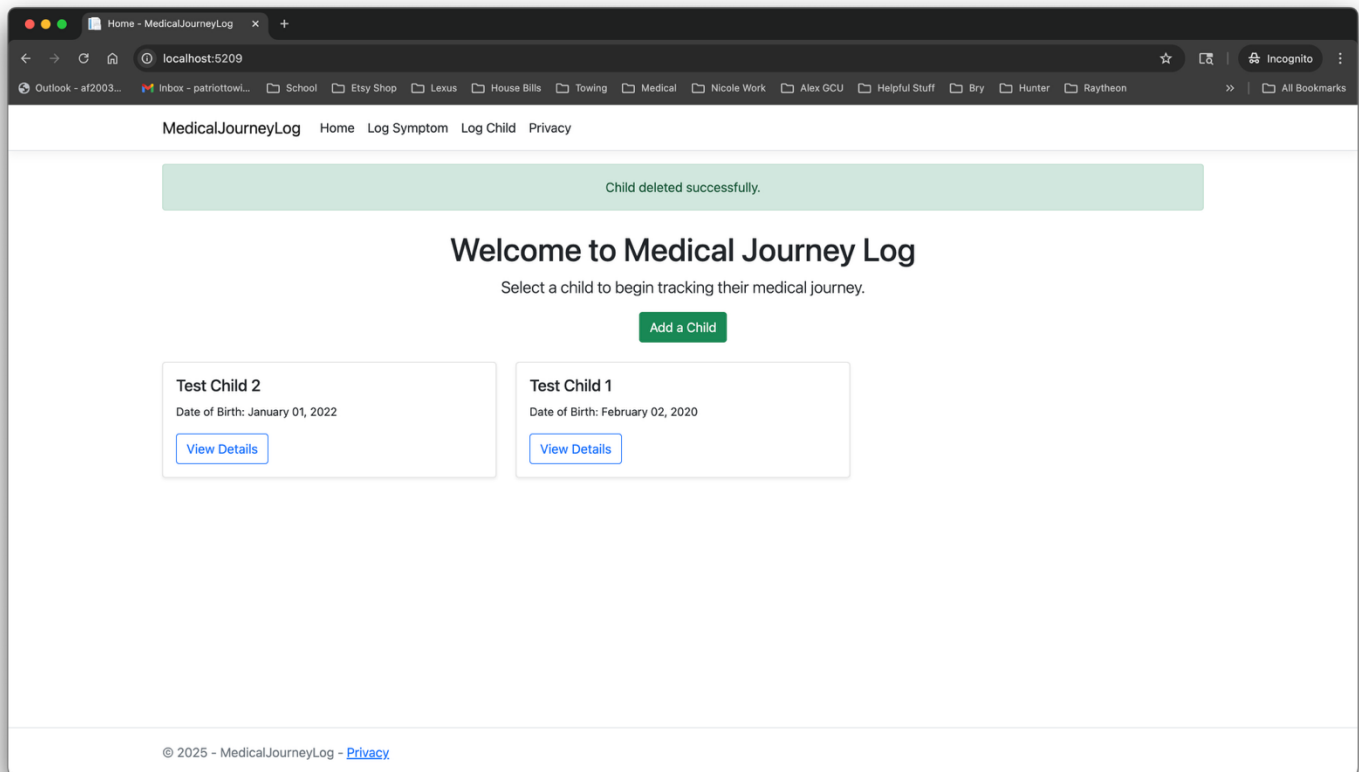


Figure 4 Screenshot showing successful deletion of a child record that has no associated symptoms. A green success message confirms the operation and returns the user to the Index page.

Delete Child (Deletion Blocked Due to Symptoms)

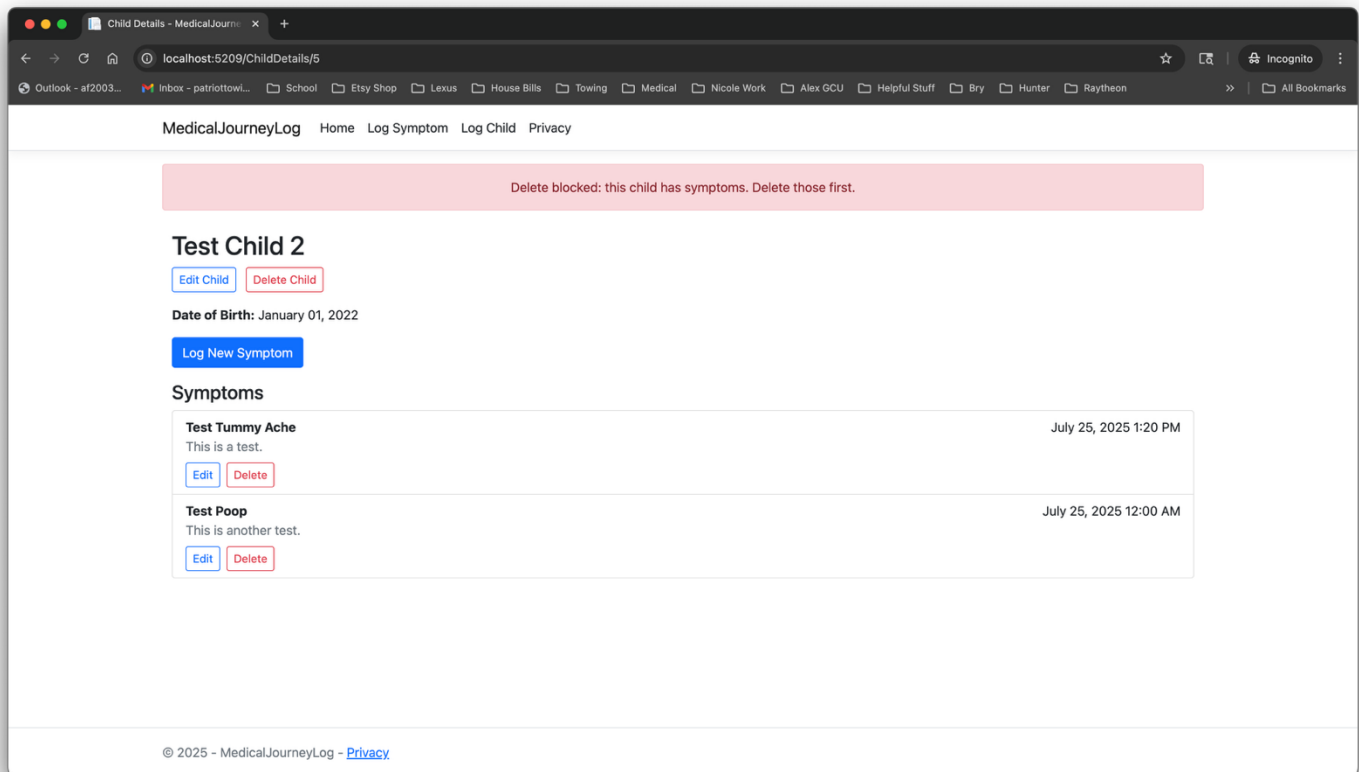


Figure 5 Screenshot showing that deletion of a child with existing symptoms is blocked. A red error alert informs the caregiver that all symptoms must be removed before the child can be deleted.

Test Case TC-007 – Edit Symptom

Field	Value
Objective	Verify caregiver can edit an existing symptom and validation prevents bad input.
Pre-Conditions / Data	Existing Symptom (Name = “Fever”) linked to a Child.
Steps (Positive)	1. Navigate to / EditSymptom /{ SymptomId } 2. Change Name to “High Fever” 3. Add Description “102°F” 4. Save.
Expected (Positive)	Redirect to / ChildDetails /{ ChildId }; success alert displayed; DB updated.
Steps (Negative A)	Clear Name field and Save.
Expected (Negative A)	Validation error “Symptom name is required.”; remain on page; DB unchanged.
Steps (Negative B)	Enter Description longer than 500 characters.
Expected (Negative B)	Validation error “Description cannot exceed 500 characters.”; remain on page; DB unchanged.
Evidence	Screenshots of success and validation error states.
Status	Pass / Fail

Edit Symptom (Before Update)

The screenshot shows a web browser window with the address bar displaying `localhost:5209/EditSymptom/6`. The browser's bookmark bar contains various links like Outlook, Inbox, School, Etsy Shop, Lexus, House Bills, Towing, Medical, Nicole Work, Alex GCU, Helpful Stuff, Bry, Hunter, Raytheon, and All Bookmarks. The page title is "Edit Symptom - MedicalJourn...". The navigation bar includes "MedicalJourneyLog", "Home", "Log Symptom", "Log Child", and "Privacy".

The main content area is titled "Edit Symptom". It contains three input fields:

- Name:** A text input field containing "Test Tummy Ache".
- Description:** A text area containing "This is a test.".
- Date Logged:** A date and time input field showing "07/25/2025, 01:20 PM" with a calendar icon on the right.

At the bottom of the form are two buttons: a blue "Save" button and a grey "Cancel" button.

The footer of the page displays "© 2025 - MedicalJourneyLog - [Privacy](#)".

Figure 6 Screenshot showing the Edit Symptom form before changes are made, displaying the existing symptom details for the selected child.

Edit Symptom (Successful Update)

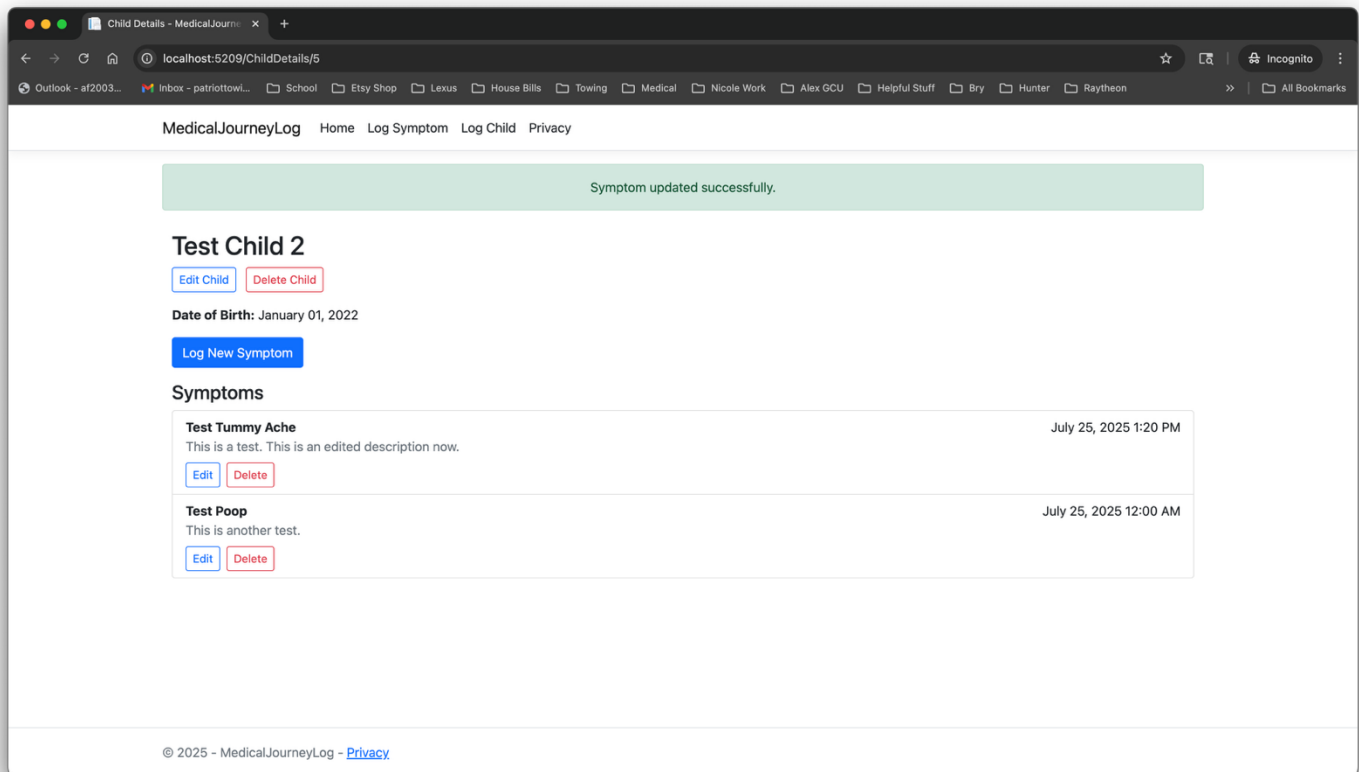


Figure 7 Screenshot showing a successful symptom update with the green success alert displayed at the top of the Child Details page. The updated symptom appears in the list.

Edit Symptom (Validation Error)

The screenshot shows a web browser window with the address bar displaying 'localhost:5209/EditSymptom/6'. The page title is 'Edit Symptom - MedicalJourneyLog'. The navigation bar includes links for 'MedicalJourneyLog', 'Home', 'Log Symptom', 'Log Child', and 'Privacy'. The main content area is titled 'Edit Symptom' and contains three input fields: 'Name', 'Description', and 'Date Logged'. The 'Name' field is empty and has a red validation error message 'Symptom name is required.' below it. The 'Description' field contains the text 'This is a test. This is an edited description now.' and the 'Date Logged' field shows '07/25/2025, 01:20 PM'. At the bottom of the form are 'Save' and 'Cancel' buttons. The footer of the page reads '© 2025 - MedicalJourneyLog - [Privacy](#)'.

MedicalJourneyLog Home Log Symptom Log Child Privacy

Edit Symptom

Name

Symptom name is required.

Description

Date Logged

[Save](#) [Cancel](#)

© 2025 - MedicalJourneyLog - [Privacy](#)

Figure 8 Screenshot showing the validation error message displayed when the required Symptom Name field is left blank. The page prevents saving until the input is corrected.

Test Case TC-008 – Delete Symptom

Field	Value
Objective	Verify caregiver can delete a symptom from a child's record.
Pre-Conditions / Data	Child with at least one Symptom in DB.
Steps	1. From ChildDetails , click Delete on a symptom 2. Confirm Delete.
Expected	Redirect to /ChildDetails/{ChildId} ; success alert displayed; symptom removed from DB.
Evidence	Screenshots before and after deletion.
Status	Pass / Fail

Delete Symptom (Before Deletion)

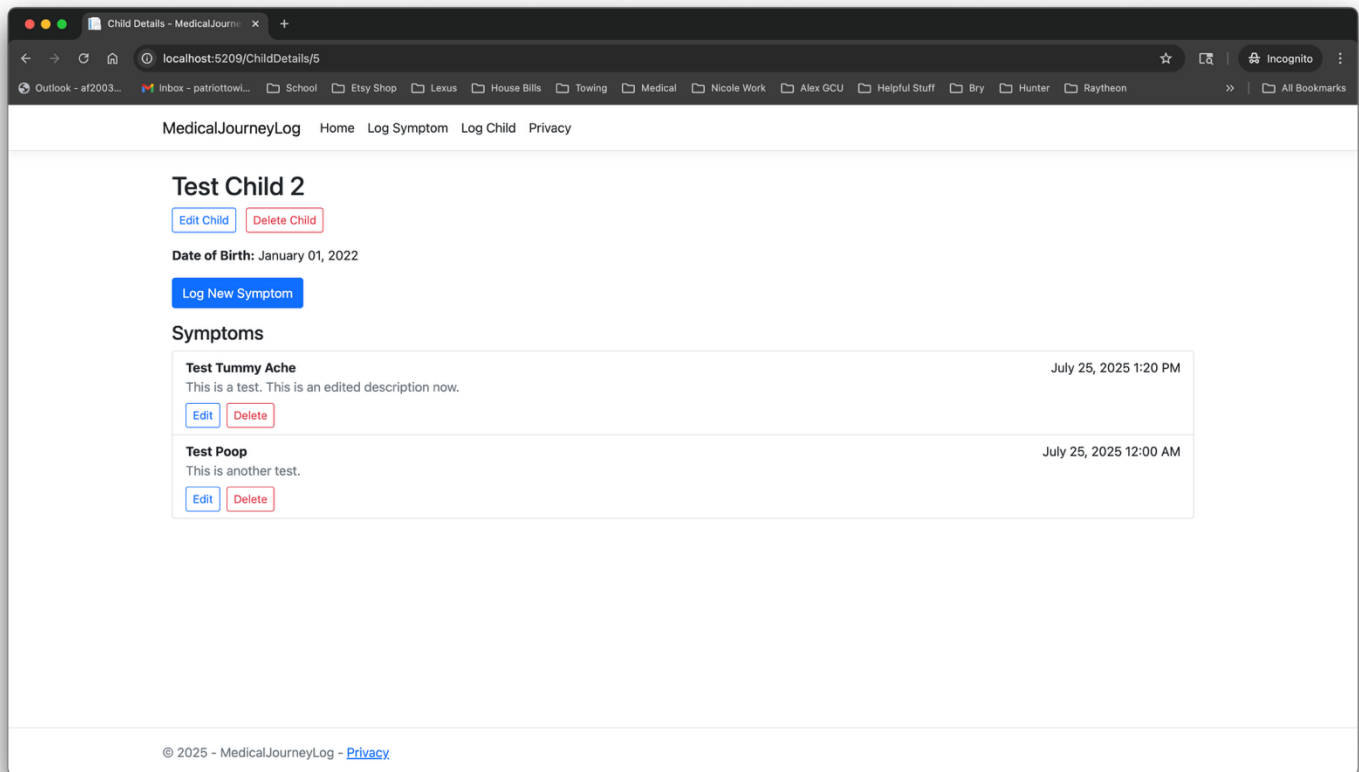


Figure 9 Screenshot showing the Child Details page before deleting a symptom. The list displays all existing symptoms with corresponding Edit and Delete controls.

Delete Symptom (Successful Deletion)

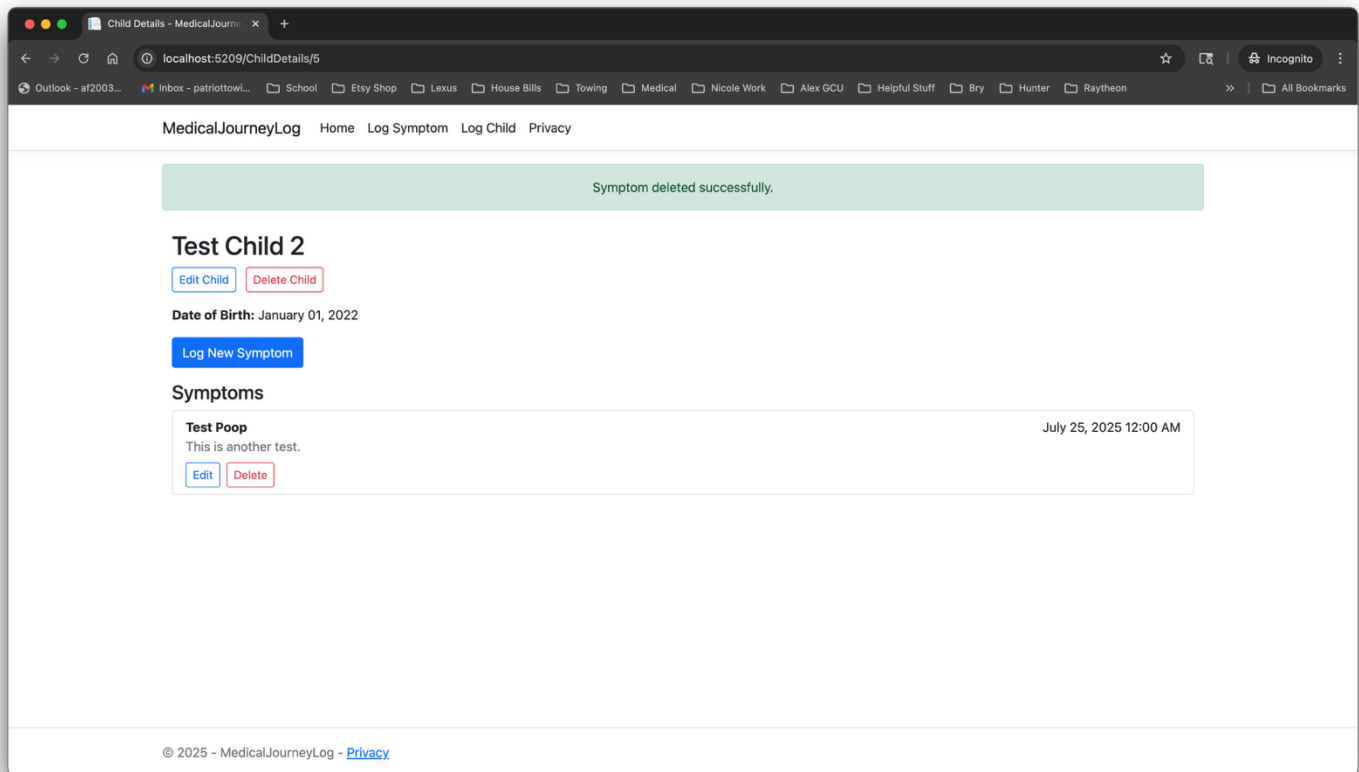


Figure 10 Screenshot showing confirmation of successful symptom deletion. The success alert appears at the top of the page, and the removed symptom no longer appears in the list.

4. Unit Test Plan

The Unit Test Plan defines how individual components (Razor Page handlers, model validation rules, and EF Core operations) are tested in isolation. Unit tests verify that methods and validation annotations behave as expected before being integrated into full workflows.

Unit Test Strategy

- **Scope:** PageModel `OnGetAsync` and `OnPostAsync` handlers for Edit/Delete (Child, Symptom), and model validation attributes in `Child.cs` and `Symptom.cs`.
- **Technique:** Manual execution with test inputs, plus review of server-side validation messages.
- **Tools:** Visual Studio debugger, browser-based form submissions, EF Core logging.
- **Responsibility:** Developer (Alex Frear).
- **Environment:** Localhost environment on MacBook Pro, .NET 7, MySQL (MAMP).

Unit Test Cases

Unit Under Test	Test ID	Description	Input	Expected Output
Child model validation	U-001	Verify Name is required	Name = ""	ModelState invalid, error "Name is required."
Child model validation	U-002	Verify Name max length 80	Name = 81-char string	ModelState invalid, error "Name cannot be longer than 80 characters."
Symptom model validation	U-003	Verify Name is required	Name = ""	ModelState invalid, error "Symptom name is required."
Symptom model validation	U-004	Verify Description max length 500	Description = 600 chars	ModelState invalid, error "Description cannot exceed 500 characters."
<code>EditChild.OnPostAsync</code>	U-005	Successful update	Valid Name + DOB	Redirects to ChildDetails with success TempData
<code>DeleteChild.OnPostAsync</code>	U-006	Block delete with symptoms	Child has ≥ 1 symptom	Redirects to ChildDetails with error TempData
<code>EditSymptom.OnPostAsync</code>	U-007	Successful update	Valid Name + Description	Redirects to ChildDetails with success TempData
<code>DeleteSymptom.OnPostAsync</code>	U-008	Successful delete	Valid SymptomId	Redirects to ChildDetails; symptom removed

Entry and Exit Criteria

- **Entry:** Component implemented and builds successfully.
- **Exit:** All unit test cases pass with expected outcomes; no critical issues remain.

5. Integration Test Plan

The Integration Test Plan validates that multiple components of the Medical Journey Log system work together correctly. This includes Razor Page handlers, EF Core data access, and MySQL persistence. The focus is on end-to-end flows across the UI, server, and database.

Integration Test Strategy

- **Scope:** Flows involving children and symptoms — creation, editing, deletion, and validation.
- **Technique:** Manual black-box testing of workflows through the web interface; verifying persisted changes in the database.
- **Tools:** Browser (Chrome), EF Core logs, MySQL Workbench.
- **Responsibility:** Developer (Alex Frear).
- **Environment:** Localhost environment on MacBook Pro, .NET 7, MySQL (MAMP).

Integration Test Cases

Test ID	Scenario	Steps	Expected Outcome
I-001	Add → Edit → Delete Child (no symptoms)	1. Log a child 2. Edit Name 3. Delete child	Success alerts shown each step; DB updated accordingly
I-002	Add Symptom → Edit → Delete	1. Log symptom for child 2. Edit symptom name 3. Delete symptom	ChildDetails updates correctly; DB updates and removes row
I-003	Delete Child with Symptoms (blocked)	1. Log child 2. Add symptom 3. Try Delete Child	Blocked with error alert; DB unchanged
I-004	Validation integration	1. Try saving child with blank Name 2. Try saving symptom with long description	Validation messages displayed; DB unchanged

Entry and Exit Criteria

- **Entry:** Unit testing complete, all pages accessible.
- **Exit:** All integration cases pass; data is consistent between UI and database; no broken workflows.

6. System Test Plan

The System Test Plan evaluates the Medical Journey Log application as a whole, ensuring that all integrated features meet the specified requirements. This testing phase validates end-to-end workflows, user experience, and system-level behaviors across the project.

System Test Strategy

- **Scope:** Full caregiver workflows — adding, editing, and deleting children and symptoms, with proper validation and error handling.
- **Technique:** Manual end-to-end testing of features in sequence, simulating real caregiver usage.
- **Tools:** Web browser (Chrome), MySQL Workbench for backend verification.
- **Responsibility:** Developer (Alex Frear).
- **Environment:** Localhost environment on MacBook Pro, .NET 7, MySQL (MAMP).

System Test Cases

Test ID	Scenario	Steps	Expected Outcome
S-001	Full lifecycle of child and symptom	1. Log a new child 2. Add symptom 3. Edit child 4. Edit symptom 5. Delete symptom 6. Delete child	All steps succeed; success/error alerts displayed; DB reflects changes
S-002	Validation enforcement	1. Attempt to log child with blank name 2. Attempt to log symptom with blank name 3. Attempt to enter >80 char name for child	Validation errors shown; no invalid DB entries
S-003	Delete guard workflow	1. Log child 2. Add symptom 3. Attempt delete child	System blocks deletion; error alert shown; DB unchanged

Entry and Exit Criteria

- **Entry:** Integration testing complete; application stable on localhost.
- **Exit:** All system test cases executed successfully; application demonstrates expected caregiver workflows without critical issues.

7. Test Case Specifications

The following specifications provide detailed definitions for the new test cases introduced in Iteration 2. Each specification includes inputs, expected outputs, and dependencies.

TC-005: Edit Child

- **Objective:** Verify caregiver can edit a child's details.
 - **Preconditions:** Child record exists in database.
 - **Inputs:** Valid and invalid values for `Name` and `DateOfBirth`.
 - **Expected Results:**
 - Valid inputs → child updated, success alert displayed, DB updated.
 - Invalid inputs (blank name, name >80 chars) → validation errors shown, DB unchanged.
 - **Dependencies:** `Child` model annotations, `EditChild` page.
-

TC-006: Delete Child (Guarded)

- **Objective:** Verify caregiver can delete a child only if no symptoms exist.
 - **Preconditions:** Case A: Child with 0 symptoms. Case B: Child with ≥ 1 symptom.
 - **Inputs:** Child ID.
 - **Expected Results:**
 - Case A → child deleted, success alert, DB row removed.
 - Case B → deletion blocked, error alert, DB unchanged.
 - **Dependencies:** `DeleteChild` page, EF Core guard check on symptoms.
-

TC-007: Edit Symptom

- **Objective:** Verify caregiver can edit an existing symptom.
 - **Preconditions:** Symptom exists for a child in DB.
 - **Inputs:** Valid and invalid values for `Name` and `Description`.
 - **Expected Results:**
 - Valid inputs → symptom updated, success alert, DB updated.
 - Invalid inputs (blank name, description >500 chars) → validation errors, DB unchanged.
 - **Dependencies:** `Symptom` model annotations, `EditSymptom` page.
-

TC-008: Delete Symptom

- **Objective:** Verify caregiver can delete an existing symptom.
- **Preconditions:** Symptom exists for a child in DB.
- **Inputs:** Symptom ID.
- **Expected Results:** Symptom deleted, success alert, removed from DB and no longer visible on Child Details.
- **Dependencies:** `DeleteSymptom` page, EF Core deletion.

8. Test Analysis Approval Form

The following approval form documents that the test analysis for Iteration 2 has been completed. The listed features have been implemented, tested, and reviewed according to the CST-452 Milestone 4 requirements.

Test Summary

Feature ID	Description	Test Case(s)	Status
F1	Edit Child	TC-005	Pass
F2	Delete Child (guarded)	TC-006	Pass
F3	Edit Symptom	TC-007	Pass
F4	Delete Symptom	TC-008	Pass
F5	Validation Enforcement	TC-005 to TC-008	Pass

9. Source Code Listing – Iteration 2

The following code listings represent the new and updated components implemented in Iteration 2. These files extend the base functionality delivered in Iteration 1, adding editing, deletion, and validation features for both children and symptoms.

Updated Model Files

Child.cs

```
using System.ComponentModel.DataAnnotations;

namespace MedicalJourneyLog.Models
{
    public class Child
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Name is required.")]
        [StringLength(80, ErrorMessage = "Name cannot be longer than 80 characters.", AllowEmptyStrings = false)]
        public string Name { get; set; } = string.Empty;

        [DataType(DataType.Date)]
        [Display(Name = "Date of Birth")]
        public DateTime? DateOfBirth { get; set; }

        public ICollection<Note>? Notes { get; set; }
        public ICollection<Symptom>? Symptoms { get; set; }
        public ICollection<Milestone>? Milestones { get; set; }
        public ICollection<Appointment>? Appointments { get; set; }
        public ICollection<CaregiverChild>? CaregiverChildren { get; set; }
    }
}
```

```
using System.ComponentModel.DataAnnotations;

namespace MedicalJourneyLog.Models
{
    public class Symptom
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Symptom name is required.")]
        [StringLength(80, ErrorMessage = "Symptom name cannot be longer than 80 characters.")]
        public string Name { get; set; } = string.Empty;

        [StringLength(500, ErrorMessage = "Description cannot exceed 500 characters.")]
        public string? Description { get; set; }

        [DataType(DataType.DateTime)]
        [Display(Name = "Date Logged")]
        public DateTime DateLogged { get; set; } = DateTime.Now;

        [Required]
        public int ChildId { get; set; }
        public Child? Child { get; set; }
    }
}
```

New Razor Pages

EditChild.cshtml.cs

```
using MedicalJourneyLog.Data;
using MedicalJourneyLog.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

namespace MedicalJourneyLog.Pages
{
    public class EditChildModel : PageModel
    {
        private readonly ApplicationDbContext _context;
        public EditChildModel(ApplicationDbContext context) => _context = context;

        [BindProperty]
        public Child Child { get; set; } = new();

        public async Task<IActionResult> OnGetAsync(int id)
        {
            var child = await _context.Children.FirstOrDefaultAsync(c => c.Id == id);
            if (child is null) return NotFound();
            Child = child;
            return Page();
        }

        public async Task<IActionResult> OnPostAsync(int id)
        {
            if (!ModelState.IsValid) return Page();

            var childToUpdate = await _context.Children.FirstOrDefaultAsync(c => c.Id ==
id);
            if (childToUpdate is null) return NotFound();

            childToUpdate.Name = Child.Name;
            childToUpdate.DateOfBirth = Child.DateOfBirth;

            await _context.SaveChangesAsync();

            TempData["SuccessMessage"] = "Child updated successfully.";
            return RedirectToPage("/ChildDetails", new { id });
        }
    }
}
```

DeleteChild.cshtml.cs

```
import React from 'react';

const Card = (using MedicalJourneyLog.Data;
using MedicalJourneyLog.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

namespace MedicalJourneyLog.Pages
{
    public class DeleteChildModel : PageModel
    {
        private readonly ApplicationDbContext _context;
        public DeleteChildModel(ApplicationDbContext context) => _context = context;

        [BindProperty]
        public Child Child { get; set; } = new();

        public string? BlockReason { get; set; }

        public async Task<IActionResult> OnGetAsync(int id)
        {
            var child = await _context.Children
                .Include(c => c.Symptoms)
                .FirstOrDefaultAsync(c => c.Id == id);

            if (child is null) return NotFound();

            Child = child;

            if (child.Symptoms != null && child.Symptoms.Any())
            {
                BlockReason = "This child has one or more symptoms. Please delete the symptoms first
before deleting the child.";
            }

            return Page();
        }

        public async Task<IActionResult> OnPostAsync(int id)
        {
            var child = await _context.Children
                .Include(c => c.Symptoms)
                .FirstOrDefaultAsync(c => c.Id == id);

            if (child is null) return NotFound();

            if (child.Symptoms != null && child.Symptoms.Any())
            {
                TempData["ErrorMessage"] = "Delete blocked: this child has symptoms. Delete those
first.";
                return RedirectToPage("/ChildDetails", new { id });
            }

            _context.Children.Remove(child);
            await _context.SaveChangesAsync();

            TempData["SuccessMessage"] = "Child deleted successfully.";
            return RedirectToPage("/Index");
        }
    }
}

rops) => {
    return (
        <div className="card mb-4" style={{ width: '18rem' }}>
            <img src={props.imgURL} className="card-img-top" alt={props.albumTitle} />
            <div className="card-body">
                <h5 className="card-title">{props.albumTitle}</h5>
                <p className="card-text">{props.albumDescription}</p>
                <button className="btn btn-primary">{props.buttonText}</button>
            </div>
        </div>
    );
};

export default Card;
```

```

using MedicalJourneyLog.Data;
using MedicalJourneyLog.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

namespace MedicalJourneyLog.Pages
{
    public class EditSymptomModel : PageModel
    {
        private readonly ApplicationDbContext _context;
        public EditSymptomModel(ApplicationDbContext context) => _context = context;

        [BindProperty]
        public Symptom Symptom { get; set; } = new();

        public async Task<IActionResult> OnGetAsync(int id)
        {
            var symptom = await _context.Symptoms.FirstOrDefaultAsync(s => s.Id == id);
            if (symptom is null) return NotFound();
            Symptom = symptom;
            return Page();
        }

        public async Task<IActionResult> OnPostAsync(int id)
        {
            if (!ModelState.IsValid) return Page();

            var symptomToUpdate = await _context.Symptoms.FirstOrDefaultAsync(s => s.Id ==
id);
            if (symptomToUpdate is null) return NotFound();

            symptomToUpdate.Name = Symptom.Name;
            symptomToUpdate.Description = Symptom.Description;
            symptomToUpdate.DateLogged = Symptom.DateLogged;

            await _context.SaveChangesAsync();

            TempData["SuccessMessage"] = "Symptom updated successfully.";
            return RedirectToPage("/ChildDetails", new { id = symptomToUpdate.ChildId });
        }
    }
}

```

```
using MedicalJourneyLog.Data;
using MedicalJourneyLog.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

namespace MedicalJourneyLog.Pages
{
    public class DeleteSymptomModel : PageModel
    {
        private readonly ApplicationDbContext _context;
        public DeleteSymptomModel(ApplicationDbContext context) => _context = context;

        [BindProperty]
        public Symptom Symptom { get; set; } = new();

        public async Task<IActionResult> OnGetAsync(int id)
        {
            var symptom = await _context.Symptoms.FirstOrDefaultAsync(s => s.Id ==
id);
            if (symptom is null) return NotFound();
            Symptom = symptom;
            return Page();
        }

        public async Task<IActionResult> OnPostAsync(int id)
        {
            var symptom = await _context.Symptoms.FirstOrDefaultAsync(s => s.Id ==
id);
            if (symptom is null) return NotFound();

            var childId = symptom.ChildId;

            _context.Symptoms.Remove(symptom);
            await _context.SaveChangesAsync();

            TempData["SuccessMessage"] = "Symptom deleted successfully.";
            return RedirectToPage("/ChildDetails", new { id = childId });
        }
    }
}
```

Updated Razor Page

ChildDetails.cshtml

```
@page "{id:int}"
@model MedicalJourneyLog.Pages.ChildDetailsModel
@{
    ViewData["Title"] = "Child Details";
}

<div class="container mt-4">
    @if (TempData["SuccessMessage"] is string ok && !string.IsNullOrEmpty(ok))
    {
        <div class="alert alert-success">@ok</div>
    }
    @if (TempData["ErrorMessage"] is string err && !string.IsNullOrEmpty(err))
    {
        <div class="alert alert-danger">@err</div>
    }

    <h2>@Model.Child.Name</h2>

    <div class="mb-3">
        <a asp-page="/EditChild" asp-route-id="@Model.Child.Id" class="btn btn-outline-primary btn-sm">Edit Child</a>
        <a asp-page="/DeleteChild" asp-route-id="@Model.Child.Id" class="btn btn-outline-danger btn-sm">Delete Child</a>
    </div>

    <p>
        <strong>Date of Birth:</strong>
        @Model.Child.DateOfBirth?.ToString("MMMM dd, yyyy")
    </p>

    <a class="btn btn-primary mb-3"
        asp-page="/LogSymptom"
        asp-route-childId="@Model.Child.Id">
        Log New Symptom
    </a>

    <h4>Symptoms</h4>

    @if (Model.Child.Symptoms != null && Model.Child.Symptoms.Any())
    {
        <ul class="list-group">
            @foreach (var symptom in Model.Child.Symptoms)
            {
                <li class="list-group-item">
                    <div class="d-flex justify-content-between align-items-start">
                        <div>
                            <strong>@symptom.Name</strong>
                            @if (!string.IsNullOrEmpty(symptom.Description))
                            {
                                <div class="text-muted">@symptom.Description</div>
                            }
                        </div>
                        <div class="text-end">
                            <time datetime="@symptom.DateLogged.ToString("o")">
                                @symptom.DateLogged.ToString("MMMM dd, yyyy h:mm tt")
                            </time>
                        </div>
                    </div>
                    <div class="mt-2 d-flex gap-2">
                        <a asp-page="/EditSymptom"
                            asp-route-id="@symptom.Id"
                            class="btn btn-outline-primary btn-sm">Edit</a>

                        <form method="post"
                            asp-page="/DeleteSymptom"
                            asp-route-id="@symptom.Id"
                            onsubmit="return confirm('Delete this symptom?');">
                            <button type="submit" class="btn btn-outline-danger btn-sm">Delete</button>
                        </form>
                    </div>
                </li>
            }
        </ul>
    }
    else
    {
        <p>No symptoms recorded yet.</p>
    }
</div>
```